SM12x Inference Optimization with vLLM

Meng Wang | DevTech Compute APAC, NVIDIA

2026-08-23

Agenda

Frontier Models on SM12x

Up the vLLM Stack

Splitting Prefill and Decode

The Gate, and the Community

Frontier Models on SM12x

Frontier Models · Demand

The models people already want on these cards

What the field looks like in 2026

Everything interesting is **MoE**, and the active fraction is small — **3–7%** of total parameters. A 744B model does A40B of work per token.

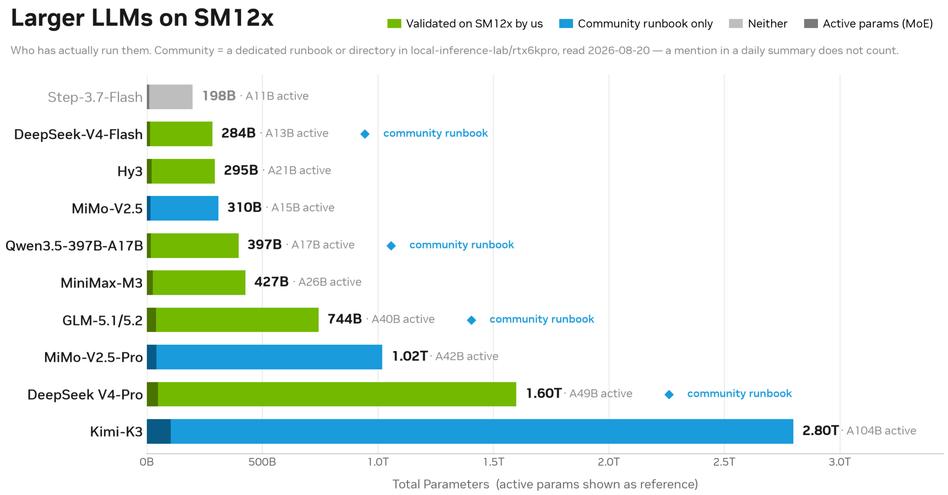
Half the field moved to **sparse or windowed attention**, which changes what decode reads per token more than any kernel does.

And the blue rows are the point

Of the models **we** have not validated, **three of four** have a community runbook. Kimi-K3's is **56 files** with its own validation directory — for a **2.8T** model, on PCIe cards.

That work is in a **863★** field wiki and an **Apache-2.0** kernel library, both of which get their own slides at the end.

People are already serving these models on these cards. So the question is not **whether** — it is **what the card's shape does to you**, and where in vLLM you deal with it.



Green = us · blue = only the community has run it

Frontier Models · The Cards

One architecture (CC 12.0/12.1), several very different deployment envelopes

SKU	SMs	Memory	Power	Interconnect
RTX PRO 5000 Blackwell	110	72 GB GDDR7 (48 GB var.)	350 W	**PCIe Gen5**
RTX PRO 6000 Blackwell (WS / Server)	188	96 GB GDDR7	600 W	PCIe Gen5
GeForce RTX 5090	170	32 GB GDDR7	575 W	PCIe Gen5
DGX Spark (SM121)	—	128 GB unified LPDDR5x	—	NVLink-C2C

Shared by every discrete card: Blackwell tensor cores with FP4 · GDDR7 · PCIe Gen5 — and **no NVLink between cards**.
Watch **sustained** clocks: the 350 W part throttles under tensor-heavy serving where the 600 W parts barely do.

Frontier Models · Three Ratios

Three numbers off that table decide everything downstream

Compute-rich

FP4 ridge on the RTX PRO 5000: **~800 op/B** (FP8: ~400).
Work below the ridge is memory-bound — and the ridge is **high**.

Bandwidth-modest

1.3–1.8 TB/s GDDR7 against **3–4×** that on HBM parts.
Weight streaming in decode is the structural bottleneck.

Comm-poor

Collectives run on PCIe: all-reduce busbw lands **an order of magnitude** under NVLink-class fabrics.
Every TP rank added must pay for itself.

Read right to left, these are the talk: **comm is the tax · decode streams weights · FP4 is compute the kernel has to reach**. Each lands on a different layer of vLLM.

Everything after this came out of a roofline model of exactly these three numbers, or out of a profiler — reported as **bands**, because without kernel traces nobody can say how much overlaps.

Up the vLLM Stack

vLLM Stack · The Map

Three problems, five layers — this table is the rest of the talk

Layer	What it owns	Problem it answers	Where it landed
Distributed	TP · SP · async-TP · all-reduce	**Comm is the tax**	vLLM#47979 · FlashInfer#4393
Kernel backends	Attention · MoE GEMM	**FP4 needs the kernel**	DeepGEMM#324 · FlashInfer#3395 · vLLM#52016
Weights / memory	Expert offload · KV	**Decode streams weights**	vLLM#51710
Scheduler	Pacing · offline planning	All three contend	vLLM#51710
Deployment	P/D fleets · KV connector	All three at once	vLLM#51538

Two of these five rows are **not vLLM code** — they are kernels vLLM **selects**. That is how the stack works, and it is why a serving-side talk has to keep going down a layer.

vLLM Stack · Distributed

vLLM#47979 — SP/async-TP for CC 12.0, over a custom all-reduce backend

Why comm is a floor

A ring all-reduce moves **2(N−1)/N** of the activation per rank, and that does **not** shrink with ranks — unlike weights, which shard 1/N.
Compute divides. Communication does not.

What vLLM changed

SP / async-TP thresholds for **CC 12.0**, symmetric-memory gating, topology-safe barriers.
FP8 MoE, TP4, vs NCCL: throughput **+12%**, TTFT **−27%**, ITL −12% — **9 h soak clean**.

The collective underneath

FlashInfer#4393: staged CUDA-IPC pushes, 4+4 island decomposition, tuned per (TP, hidden, batch).
1.2–5.9× vs NCCL at kernel level, TP2–8, 8-GPU PCIe box.

NCCL is built for **NVLink-class fabrics** — it is aimed elsewhere, not wrong. On PCIe, **topology- and size-aware** collectives claw the deficit back one message class at a time.

vLLM Stack · FP4 GEMM

DeepGEMM#324 (merged) + #380 — via vLLM backend selection

Dense GEMM (% of roofline @ the measured clock)

FP8 **96%** · FP4 **96%** · BF16 ~98%

Grouped / MoE — and the two ceilings

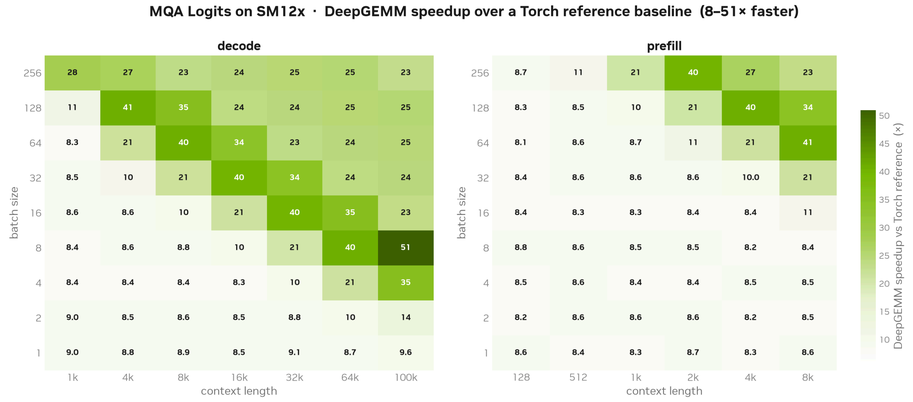
M-grouped **prefill** chases the **FLOP peak** — compute-bound.

#380 **decode** chases the **DRAM peak**: skip-M-padding, BLOCK_M=32, warp-cooperative `cp.async` — **98–99% of DRAM peak**, beating CUTLASS on every tested shape.

Coverage

Dense · grouped · K-grouped EP · batched einsum · MQA logits (FP8/FP4, paged).

Two ceilings, both reached. The roofline said which one each kernel was fighting **before** it was written — that is the whole return on modelling first.



MQA logits vs a Torch reference — 8–51×

NVIDIA

vLLM Stack · Attention

FlashInfer#3395 (merged) — dual cache · TMA gather

Semantics the hardware will not express

Per-token V-scale folded in by hand; **dual cache** with two different page geometries.

Survivors, and what evidence killed

Kept: **TMA gather** · warp-specialized IO/math split · MG 2× KV reuse.

Rejected by measurement: 2-CTA, cluster/DSM, block-scaled XV, split kernels. The rejected list is where the architecture taught us something.

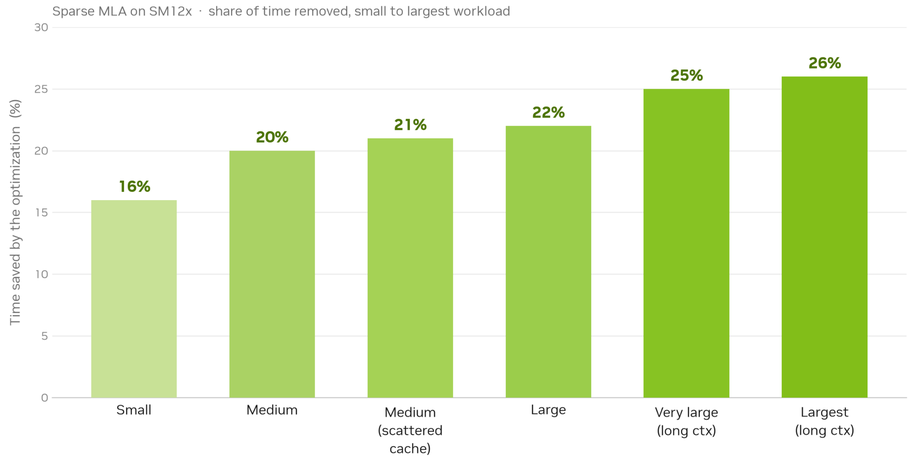
Results

1.19–1.34× over our previous version, growing with workload.

4–9× over a legacy Triton reference — **that baseline was unoptimized**, and this deck says so.

The profiler named the limiter and it was **L1/TEX traffic from the gather** — not DRAM, not the tensor pipe. **The gather is the kernel.** Profile before you optimize: the saturated pipe is rarely the assumed one.

The harder the workload, the bigger the win



Time removed by this optimization round

NVIDIA

vLLM Stack · Weights

The tax vanishes past a critical tokens-per-forward

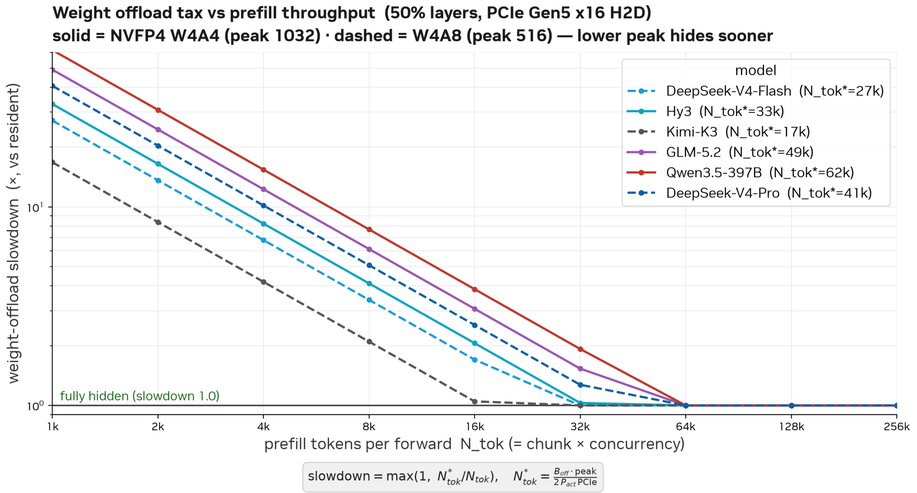
Prefill streams weights **once per forward** — so enough tokens per forward amortize the copy to zero.

$N^*_{\text{tok}} = \frac{B_{\text{off}} \cdot \text{peak}}{2P_{\text{act}} \cdot \text{H2D}}$ — **invariant in TP**.

DSv4-Flash: hidden from **~27k tokens** · Hy3 ~33k — even a 2.8T model only needs ~34k.

Landing in vLLM#51710: a 1.6T model at TP8 goes from **cannot start** to serving; pacing the H2D copies against the collectives adds **+11% req/s**.

Capacity stops being the prefill gate — a 72 GB card prefills what it cannot hold.
Still open: nobody has swept tokens-per-forward and watched the tax cross zero at the predicted point. It is a prediction, not a result.



50% of layers offloaded · measured H2D bandwidth

vLLM Stack · Scheduler

vLLM#51710 — the layer that sees all three problems contending at once

Collective-aware pacing

H2D weight copies and TP collectives contend for the **same PCIe**. Scheduled blind, they collide.

Paced against each other: **+11% req/s**, no kernel touched.

Semantic expert selectors

Which experts stay resident and which get offloaded, chosen from **what the model actually routes** — not from a fixed rule.

Offline schedule planner

Serve **once** with a manifest, then score every candidate schedule **offline**.

The alternative is re-benchmarking the configuration space every time the SLA moves.

On a card where comm is a floor and weights are in flight, the scheduler is the **only layer that sees all the contention at once**. A kernel cannot fix a scheduling collision.

Splitting Prefill and Decode

P/D Split · Why

Three independent arguments arriving at the same deployment shape

They fight

One instance runs a **compute-bound** prefill and a **bandwidth-bound** decode against the same SMs, fabric and scheduler.

TTFT and TPOT stop being independently tunable.

Opposite parallel plans

Prefill wants **TP**: shard compute, pay the all-reduce — about **a third of kernel time** in a profiled TP4 prefill.

Decode wants **DP/EP**: shard requests and experts, skip the all-reduce.

Opposite silicon

Prefill is where **FP4 compute** pays.

Decode is a **weight stream** and wants bandwidth. One card cannot be the right answer to both.

Not slideware: vLLM#51538 validates P/D end to end over a KV connector — and the accuracy gate **holds through the handoff** (GSM8K **94.9%**), which is the part a KV transfer could quietly break.

P/D Split · Measured

2026-08-10 · 4 GPUs per config · ISL 8000 / OSL 1000 · speedup vs IFB at equal GPU count · hybrid decode runs Humming (open-source operators)

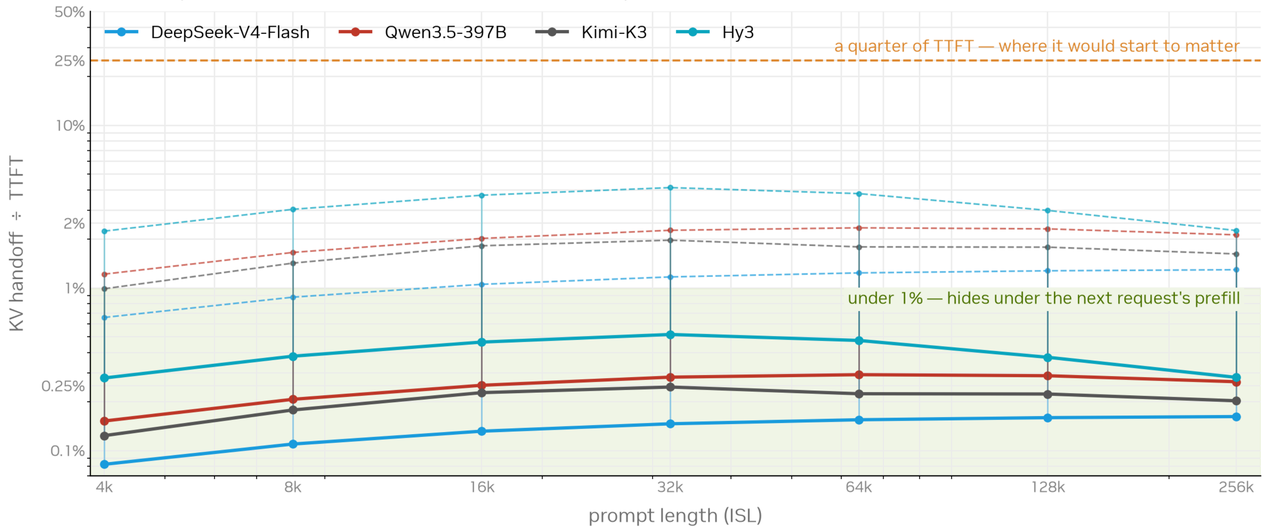
Model	TTFT	TPOT	5000 P + H20 D	all RTX PRO 5000
DeepSeek-V4-Flash	1.5 s	10 ms	**3.06×**	2.30×
		20 ms	**1.76×**	1.57×
		30 ms	**1.53×**	1.43×
GLM-5.1	3 s	20 ms	**3.17×**	1.18×
		50 ms	**1.49×**	1.19×
Qwen3.5-397B	3 s	20 ms	**1.24×**	1.07×
		50 ms	**1.17×**	0.94× — loses

The hybrid leads at every SLA measured — and by the most where the SLA is tightest, which is the decode argument again: the tighter the TPOT, the more the decode side wants bandwidth this card does not have.
And keep the counter-example: Qwen at 50 ms all-SM12x scores **0.94×** — it **loses** to in-flight batching. Splitting a decode side that cannot keep up just moves the bottleneck.

P/D Split · What It Costs

The KV handoff, priced by the model — and fleet sizing, measured per model

TP8 · PCIe Gen5, no NVLink · solid = a link per GPU, dashed = one shared uplink; the band is the topology nobody has measured
at 32k: DeepSeek 0.15–1.2% Qwen3.5 0.28–2.3% Kimi 0.25–2.0% Hy3 0.52–4.1%



Bytes and lanes both come from the two sides' parallel plans — a replicated MLA cache ships once, not once per rank. Transfer in isolation: it shares the fabric with the prefill instance's own all-reduce (33% of prefill kernel time, measured), and that contention is a scheduling problem nothing here measures.

Fleet sizing — prefill instances one decode instance can feed:

DeepSeek-V4-Flash **3:1** · GLM-5.1 **2.5:1** · Qwen3.5-397B **1:1**
(hybrid fleet; all-SM12x is 2:1 / 1:1 / 2:3)

Qwen is inverted — more decode than prefill, because decode is its weak side. Same fact as its 0.94×.

The handoff is **a small fraction of TTFT and flat in prompt length** — not luck: the sparse KV that makes it cheap is what makes a model good on this card.

Guessing **1:1** fleets oversubscribes DSv4-Flash's prefill fleet **3×**. **Not modelled:** contention with the prefill instance's own all-reduce.

The Gate, and the Community

The Gate · Accuracy

Every speed number in this talk is gated on quality, not just on throughput

The bring-up, and its gate

vLLM#43477: sparse MLA via FlashInfer, MoE via DeepGEMM, + spec decoding and CUDA graphs.

GSM8K **95.0%** · GPQA-D **73.2%** → **87.4%** with thinking, and a prebuilt image — **reproducible, not described**.

Why it is not optional here

On this card nearly every performance lever — **FP4**, sparse attention, speculative decoding, a **KV handoff across machines** — is also a lever that can change what the model says.

Any of them shipped without a gate is a liability, not a speedup.

And the gate holds **through** the P/D handoff — GSM8K **94.9%** on the disaggregated path, which is exactly the thing a KV transfer could break quietly.

Community · Field Wiki

local-inference-lab/rtx6kpro — Martin Vit + the Discord

863★ · 57 forks · 962 files, opened March 2026 and pushed the day these numbers were read.

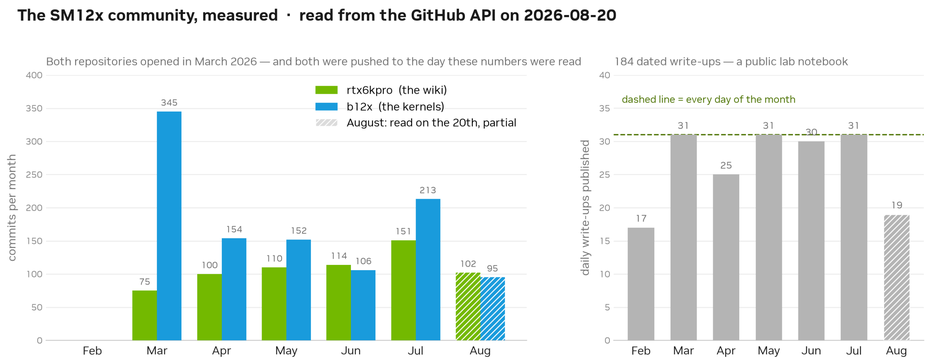
124 model runbooks, and they are **versioned** — `ds4-flash` v1→v6, `ds4dspark` r2→r18. Recipes with a history, not a snapshot.

184 dated write-ups since 2026-02-12: a public lab notebook, most months every single day.

Docker pinned to **CUDA 13.3**, build scripts in-repo; topology guides per named chassis (ESC8000A-E13P, ASRockRack, WRX90).

Their primary documented engine is **vLLM**.

And they benchmark us. `results_nvidia_nvfp4.json` sits in that repo next to `results_lukealonso_nvfp4.json` and `results_awq.json` — raw JSON, MTP and no-MTP, committed. Not a screenshot of a table.



Both repos, read from the GitHub API on 2026-08-20

Community · The Kernels

local-inference-lab/b12x — Apache-2.0, Luke Alonso · CuTe-DSL kernels for SM120/SM121

A real engineering artifact

182★ · 1065 commits · 679 files, of which **213 are tests**.

Collectives that **compress**: **48.4% fewer wire bytes** than BF16 — and **island reduce-scatter**, which our own custom all-reduce reached independently.

They ported DeepGEMM

Dense MXFP8 went **1.44× slower** → **~1.6× faster** than FlashInfer CUTLASS — their shape, a Pro 6000.

The whole win was **tile selection**: output byte-identical, cos 1.0, maxabs 0.

And now a vLLM backend

vLLM#52016 **merged 2026-08-14**: dense linear — FP8, 128×128 block FP8, MXFP8, NVFP4, MXFP4. Attention (#52017) and FP4 MoE (#52018) in review.

First-party, not via FlashInfer: `pip install vllm[b12x]`, pure Python, **no build step**.

Why it merged in two days: it uses vLLM's **existing** backend interfaces — no new abstraction, no model-runner changes — and slots into automatic selection **after** the tuned backends and **before** emulation. A 7700-line monolithic PR was closed and **split into three** first.
And the discipline is the part to copy: their port postmortem carries a dated self-correction — a claim had been **asserted, never measured**, a re-audit found a dropped re-quantization worth ~2.7× excess weight error, and they left the correction in the document.

SM12x with vLLM, in One Line

One card shape · three problems · and the layer that answers each





The card

- FP4-rich, BW-modest, PCIe-only
- Read the ratios before writing code

Distributed

- Comm is a floor you hide, not work you shard
- SP/async-TP + a PCIe-aware collective

Backends

- FP4 is compute the kernel must reach
- Two ceilings — profile to learn which

Memory + sched

- Decode moves bytes — so move them
- Offload past N*, then pace the copies

Deployment

- SM12x prefill + HBM-class decode
- Handoff is cheap; fleet ratio is per model